



RELATÓRIO DE QUALIDADE E DESEMPENHO DA API

Mariana Rebelo Tebecherani

São José dos Campos
2026

SUMÁRIO:

1. INTRODUÇÃO E OBJETIVOS	3
2. FERRAMENTAS UTILIZADAS	3
3. METODOLOGIA DE COLETA DAS MÉTRICAS	3
4. MIDDLEWARE DE COLETA DE MÉTRICAS	4
5. RESULTADOS OBTIDOS	5
6. CONCLUSÃO	8
7. REFERÊNCIA BIBLIOGRÁFICA	9

1. INTRODUÇÃO E OBJETIVOS

Este relatório apresenta a análise de desempenho da API da aplicação Aerocode, com foco nas métricas de latência, tempo de processamento e tempo de resposta. Os testes foram realizados utilizando o Apache JMeter para simulação de carga simultânea com 1, 5 e 10 usuários virtuais executando operações de consulta e CRUD sobre a aplicação.

2. FERRAMENTAS UTILIZADAS

- Apache JMeter para geração de carga e obtenção do tempo de resposta.
- Middleware customizado no backend para medição do tempo de processamento.
- Persistência das métricas em banco de dados MySQL.
- Microsoft Excel para consolidação e geração dos gráficos.

3. METODOLOGIA DE COLETA DAS MÉTRICAS

O tempo de resposta foi obtido através dos relatórios gerados pelo Apache JMeter, responsável por simular múltiplos usuários simultâneos realizando requisições HTTP contra a API da aplicação. Já o tempo de processamento foi coletado diretamente no backend através de um middleware desenvolvido especificamente para registrar o intervalo de execução de cada requisição processada pelo servidor.

As métricas coletadas pelo middleware foram persistidas em banco de dados MySQL, permitindo posterior consolidação estatística e cálculo das médias utilizadas nos gráficos.

Como o JMeter fornece o tempo total de resposta e o middleware fornece o tempo interno de processamento do servidor, a latência foi estimada matematicamente pela diferença entre o tempo de resposta e o tempo de processamento, dividindo-se o valor resultante em duas partes equivalentes, representando a latência de ida e a latência de retorno da comunicação cliente-servidor.

$$\text{Tempo de Resposta} = \text{Latência de Requisição} + \text{Tempo de Processamento} + \text{Latência de Resposta}$$

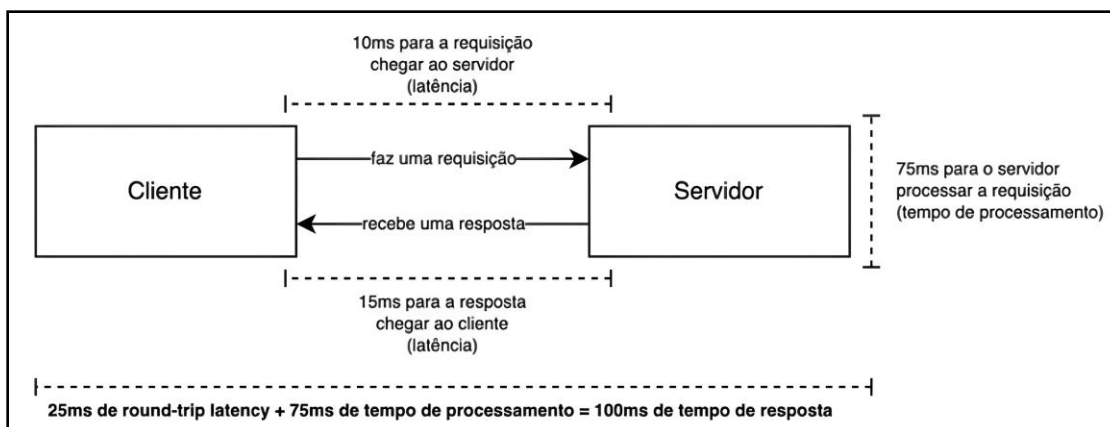


Figura 1 – Representação Conceitual das Métricas

4. MIDDLEWARE DE COLETA DE MÉTRICAS

O middleware implementado no backend foi responsável por capturar automaticamente o tempo de execução de cada requisição recebida pela API. O código utiliza a API `performance.now()` do Node.js para medir precisamente o intervalo de processamento entre o recebimento e a finalização da requisição.

```

1  import { performance } from "perf_hooks";
2  import { Request, Response, NextFunction } from "express";
3  import { env } from "../config/env";
4  import { prisma } from "../shared/prisma";
5
6  export function apiMetricsMiddleware(req: Request, res: Response, next: NextFunction) {
7      if (!env.apiMetricsEnabled) {
8          next();
9          return;
10     }
11
12     const inicio = performance.now();
13     const iniciadoEm = new Date();
14
15     res.on("finish", () => {
16         const fim = performance.now();
17         const finalizadoEm = new Date();
18         const duracaoMs = Number((fim - inicio).toFixed(2));
19         const rota = req.route?.path ? `${req.baseUrl}${req.route.path}` : req.originalUrl;
20
21         console.log(
22             `[API Metrics] ${req.method} ${req.originalUrl} ${res.statusCode} - ${duracaoMs} ms`
23         );
24
25         if (!env.apiMetricsPersist) {
26             return;
27         }
28
29         prisma.apiMetric.create({
30             data: {
31                 metodo: req.method,
32                 caminho: req.originalUrl,
33                 rota,
34                 statusCode: res.statusCode,
35                 duracaoMs,
36                 iniciadoEm,
37                 finalizadoEm,
38                 ip: req.ip,
39                 userAgent: req.header("user-agent")
40             }
41         }).catch((error) => {
42             const message = error instanceof Error ? error.message : "Erro desconhecido.";
43             console.error(`[API Metrics] Falha ao salvar metrica: ${message}`);
44         });
45     });
46
47     next();
48 }

```

Figura 2 – Middleware de coleta de métricas implementado no backend

5. RESULTADOS OBTIDOS

Os testes foram executados considerando três cenários de concorrência: 1 usuário simultâneo, 5 usuários simultâneos e 10 usuários simultâneos. Os gráficos a seguir apresentam as médias consolidadas das métricas obtidas.

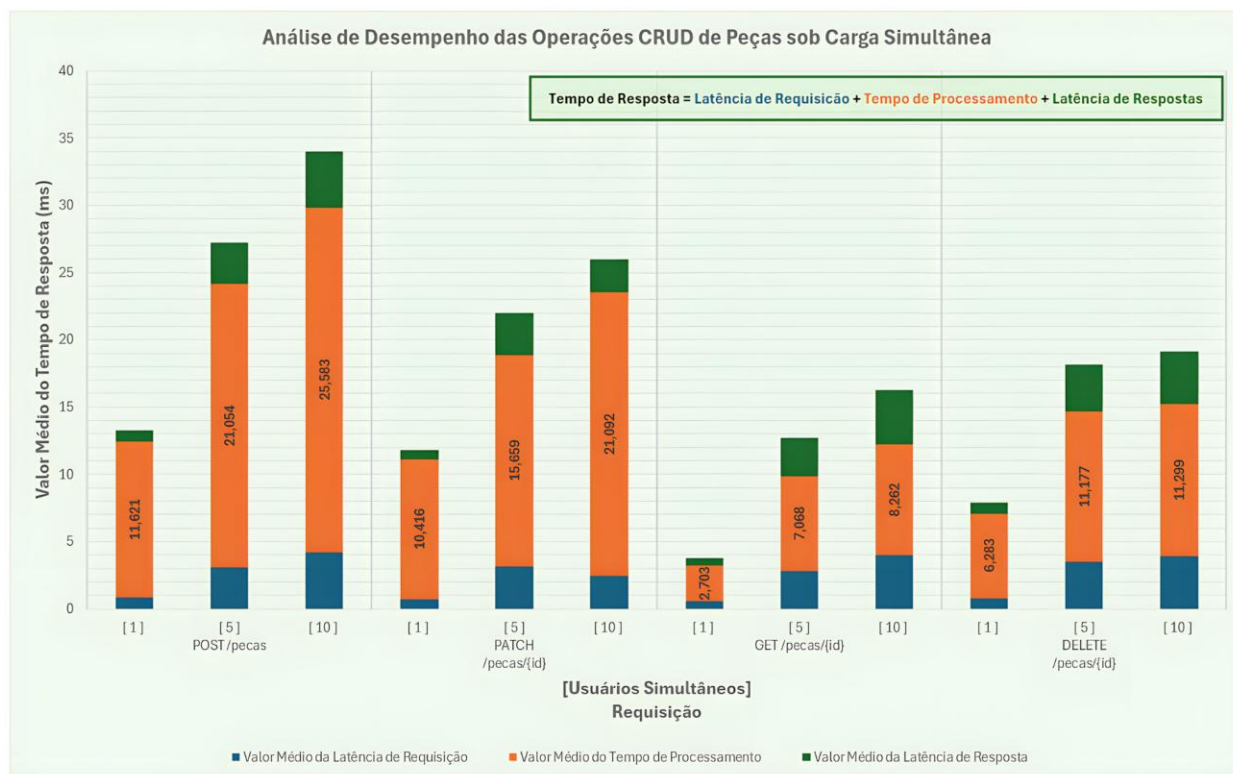


Figura 3 – Análise de desempenho das operações CRUD de peças

Observa-se que as operações de escrita (POST e PATCH) apresentaram maior tempo de processamento em comparação às operações de leitura e exclusão, comportamento esperado devido ao maior volume de validações e persistência de dados executados pelo servidor.

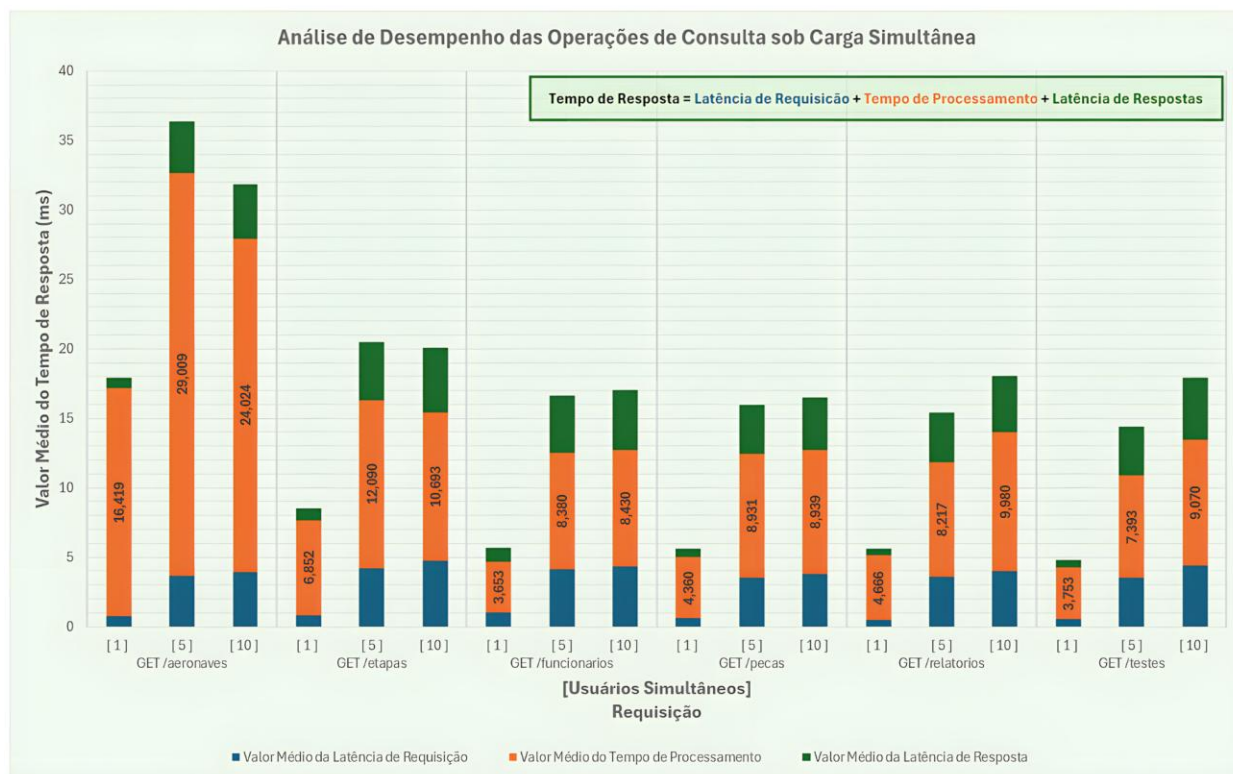


Figura 4 – Análise de desempenho das operações de consulta

As operações de consulta apresentaram comportamento estável mesmo sob aumento de carga, mantendo tempos médios de resposta reduzidos e crescimento gradual das métricas conforme o aumento do número de usuários simultâneos.

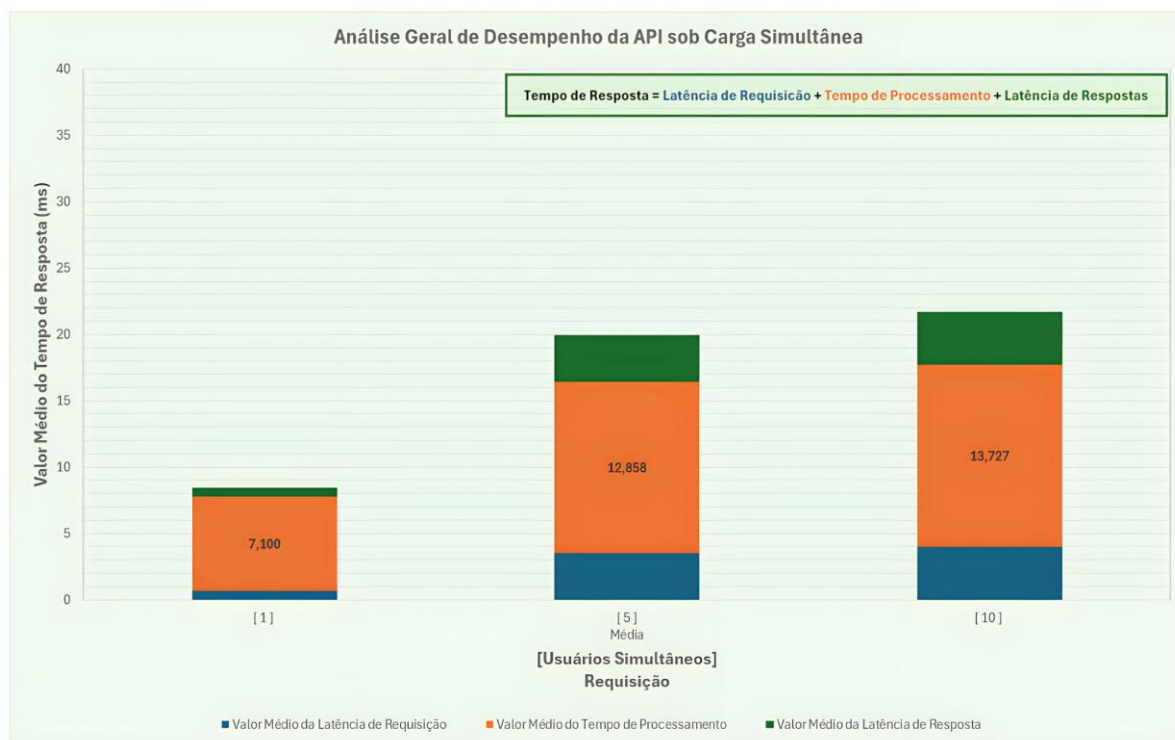


Figura 5 – Média geral de desempenho da API

A análise geral demonstra que a aplicação manteve estabilidade durante os testes de carga, sem apresentar crescimento exponencial do tempo de resposta ou degradação severa de desempenho.

6. CONCLUSÃO

Os resultados obtidos demonstram que a API da aplicação Aerocode apresentou comportamento estável e adequado durante os testes realizados. Mesmo com o aumento do número de usuários simultâneos, os tempos médios de resposta permaneceram baixos, sem crescimento exponencial ou falhas significativas.

Segundo Nielsen Norman Group, tempos de resposta abaixo de 100 ms são percebidos pelos usuários como praticamente instantâneos. Todos os cenários analisados permaneceram significativamente abaixo desse limite, indicando que a aplicação fornece uma experiência de navegação fluida e responsiva.

Tempo	Percepção
~0.1 s	Instantâneo
~1 s	Fluxo contínuo
~10 s	Perda de atenção

7. REFERÊNCIA BIBLIOGRÁFICA

NIELSEN, Jakob. Response Times: The 3 Important Limits. Nielsen Norman Group, 1993. Disponível em: <https://www.nngroup.com/articles/response-times-3-important-limits/>. Acesso em: 24 maio 2026.